The Ch(e)at Code

Agent Patterns for Copilot Chat

↑ ↑ ↓ ↓ ← → ← → B A START

ISSUE #007 • MAY 5, 2026 • 🧠 CONCEPTUAL PATTERN

You've heard this from customers: "We have AI tools everywhere now, but none of them remember what the others did." Knowledge lives in scattered chats, repos, docs, and meetings — and when someone asks a question that spans two projects, the answer doesn't exist in any single system. The knowledge fragments are all there, but no one — and no agent — can connect them.

This issue's pattern: Holographic Memory — a knowledge architecture where every node carries enough redundant context to reconstruct meaning even when other nodes are missing, giving your agent resilient cross-domain recall.

🎮 AGENT SPOTLIGHT

The Holographic Wiki

Concept by Tyson Dowd • Discussed in 1:1 (March 24, 2026)

The scenario: An enterprise runs AI-assisted workflows across multiple tools — Copilot Chat, Claude, ChatGPT, Canva, internal repos, SharePoint libraries. Each tool has its own memory. When a project manager needs to answer a cross-project question ("What design decisions from Project A affect Project B?"), no single system has the full picture. Knowledge is fragmented, tool-specific, and non-relational.

The insight: Tyson Dowd described an architecture inspired by holography — where every fragment of a hologram contains partial information about the whole image. Cut a hologram in half and you don't lose half the picture; you get a lower-fidelity version of the entire picture. Apply this to knowledge systems: every node in your knowledge graph should carry enough redundant context that losing any single node degrades fidelity but never coherence.

This isn't a specific agent build — it's an **architectural pattern** for how you design the knowledge layer underneath any agent that needs memory across tools, projects, or time.

🧩 **PATTERN BREAKDOWN**

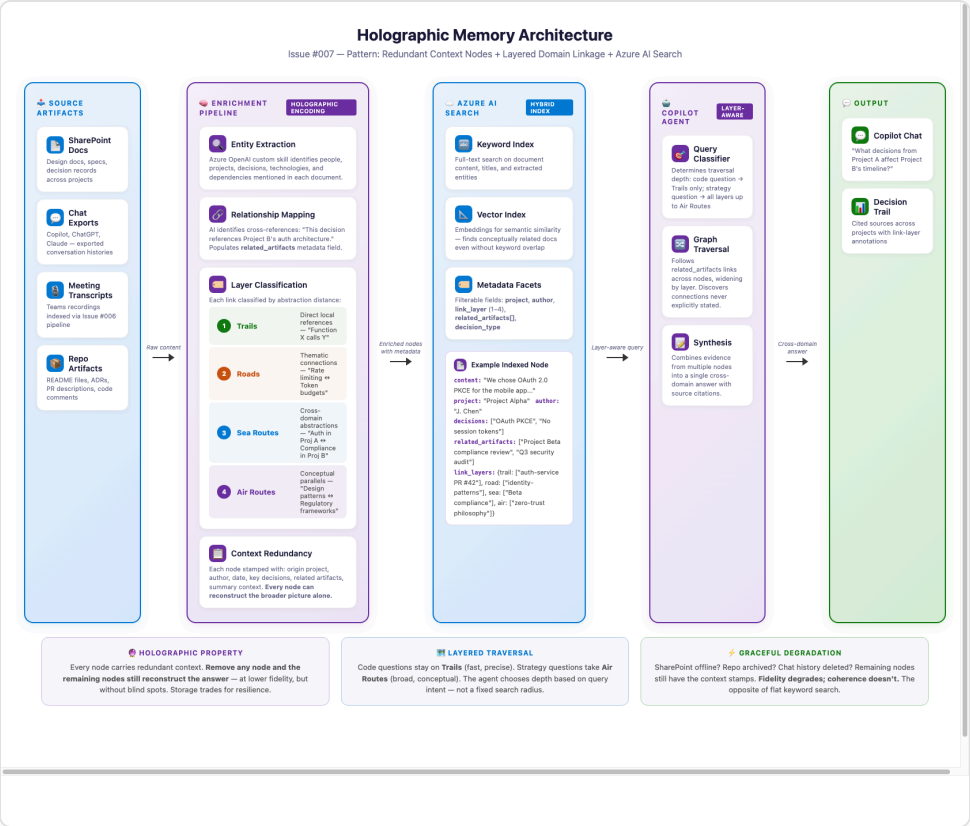
Holographic Memory

Redundant context nodes + layered domain linkage + Azure AI Search

Why this matters: Today's agent memory is flat. Copilot Chat remembers the current conversation. Custom instructions add persistence. But neither gives you *relational memory* — the ability to infer connections between concepts that were never explicitly linked. Customers building multi-agent systems or cross-project knowledge bases hit this wall immediately.

DOMAIN LINKAGE LAYERS

```
Layer 4: Air Routes — abstract conceptual links
  ↑  "Design patterns" ↔ "Regulatory frameworks"
Layer 3: Sea Routes — cross-domain abstractions
  ↑  "Auth architecture in Project A" ↔ "Compliance in Project B"
Layer 2: Roads — structured thematic connections
  ↑  "API rate limiting" ↔ "Token budget management"
Layer 1: Trails — direct local references
     "Function X calls Function Y"
```



Full pipeline: Source Artifacts → Holographic Enrichment → Azure AI Search → Layer-Aware Copilot Agent — click to enlarge

The pattern has three design decisions:

1. **Redundant context at every node.** When your agent indexes a document, a meeting summary, or a chat export, don't just store the content — store *who created it, what project it belongs to, what other artifacts it references, and what decisions it influenced*. Each node should carry enough metadata that an agent reading only that node can reconstruct the broader context. This is the "holographic" property: every piece contains a low-resolution image of the whole.

2. **Layered domain linkage, not flat tags.** Most knowledge systems use flat tags or folder hierarchies. Neither captures *abstraction distance*. Tyson's model uses four layers: **Trails** (direct code-level references), **Roads** (thematic connections within a domain), **Sea Routes** (cross-domain abstractions), and **Air Routes** (high-level conceptual parallels). When you index content, classify links by layer. This tells the agent *how far* to traverse when answering a question — a code question stays on trails; a strategy question takes air routes.

3. **Chat export as a memory layer.** Power users already chain across multiple LLMs — ChatGPT for brainstorming, Claude for code review, Copilot for integration. The key unlock: *exporting chat histories* and indexing them as knowledge artifacts. Each exported conversation becomes a node in the graph, with cross-references to the repos, docs, and decisions it touched. This turns ephemeral tool usage into durable organizational memory.

Key insight: The holographic property means your knowledge system *degrades gracefully*. If a SharePoint library goes offline, a repo is archived, or a chat history is lost, the remaining nodes still contain enough redundant context to answer questions — at lower fidelity, but without total failure. This is the opposite of how most agent memory works today, where losing a single data source creates a blind spot.

IMPLEMENTATION WITH AZURE AI SEARCH

```
Source artifacts (docs, chats, repos, meetings)
  ↓
Enrichment pipeline → extract entities, relationships, layer tags
  ↓
Azure AI Search index (hybrid: vector + keyword + metadata)
  ↓
Copilot agent queries with layer-aware retrieval
  ↓
Agent traverses graph: trails → roads → sea routes → air routes
  ↓
Response with cross-domain synthesis + source citations
```

⚠ **Design warning:** The redundant context approach increases index size significantly — expect 3–5x the storage of a naive flat index. This is a deliberate trade-off: you're spending storage to buy resilience and cross-domain recall. Size your Azure AI Search tier accordingly and use the metadata-heavy fields as filterable (not searchable) to control costs.

🛠 **TRY THIS NOW**

Build a Cross-Project Knowledge Agent This Week

1. **Find the cross-project pain.** Ask the customer: "When someone new joins a project, how long does it take them to understand what decisions were already made and why?" If the answer is "weeks" or "they ask around on Teams," you've found the scenario. PMOs, architecture review boards, and R&D leads are the sharpest entry points.

2. **Start with two data sources, not ten.** Pick one SharePoint library and one repo (or meeting archive). Index them into Azure AI Search with enriched metadata: project name, author, date, related projects, key decisions referenced. This is your minimum viable "holographic" index — two sources with cross-references.

3. **Add the "related artifacts" field.** For every document you index, populate a metadata field called `related_artifacts` with references to other documents it connects to. This is the redundant context that makes the pattern work. Use an AI enrichment step in Azure AI Search to auto-extract these relationships.

4. **Demo the cross-domain question.** Ask your agent: "What decisions from Project A could affect the timeline of Project B?" If the agent can synthesize an answer from documents that never explicitly mentioned each other, you've demonstrated the pattern. If not — you need richer metadata at indexing time.

💡 **The line that opens doors:** "What if your team could ask one question and get an answer that connects decisions across every project — even when those projects were never explicitly linked?"

🏆 **WHERE THIS PATTERN LANDS**

- **Professional services & consulting** — cross-engagement knowledge reuse, lessons learned across client projects, institutional memory for partner rotations

- **Engineering & R&D** — cross-team architecture decisions, design pattern reuse, "why did we choose X over Y" queries across repos and design docs
- **Regulated industries (pharma, finance, energy)** — compliance traceability across projects, audit trail for decisions that span multiple workstreams
- **M&A & integration** — rapidly connecting knowledge bases from two merging organizations where nothing was linked before
- **Any organization** where the answer to "why did we do it this way?" is trapped in someone's head, an old chat, or a meeting no one documented

Positioning tip: Frame this as "AI maturity, not AI novelty." Tyson's insight: we're in the "Windows 7 phase" of AI — models are stable enough that the differentiator is now *systems, memory, and scale*, not raw capability. Customers who are past the chatbot phase and asking "how do we make AI remember?" are ready for this conversation.

⚡ **QUICK TIPS**

✓ **Cross-model tool chaining is already happening.** Your customers are moving between ChatGPT, Claude, and Copilot depending on the task. The unlock: get them exporting those chat histories and indexing them. Each exported conversation becomes a knowledge node.

✓ **Azure AI Search custom skills** can auto-extract the "related artifacts" metadata at index time. Use a custom skill that calls Azure OpenAI to read each document and identify what other projects, decisions, or people it references. This automates the redundant context layer.

✓ **Pair with Issue #002 (Scoped Multi-Source Search)** for the retrieval pattern. Holographic Memory is the *indexing* architecture; Scoped Search is the *retrieval* architecture. Together, they give you a complete cross-domain knowledge agent.

Got an agent or pattern to share?

Reply to this email and you might be featured in a future issue.

The Ch(e)at Code

ABS Tech Strategy • AI Business Solutions

Questions or ideas for the next issue? Reply to this email.

Browse all issues at aka.ms/the-cheat-code

Microsoft Internal • Not for external distribution